Embeddings Vectoriales y Redes Neuronales Recurrentes

De vectores de palabras a modelos de secuencias

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule

¿Dónde Estamos?

Clase 3.1

- Texto → tokens → índices.
- Representaciones por conteo: BoW, *n*-gramas, TF-IDF.
- Embeddings preentrenados: word2vec, GloVe, BERT.

Lo que falta

Todas esas representaciones de *documento* **agregan** palabras (suma, promedio) y **pierden el orden**:

- "el perro mordió al hombre" = "el hombre mordió al perro".
- "no es bueno, es excelente" vs. "es bueno, no es excelente".

Esta clase

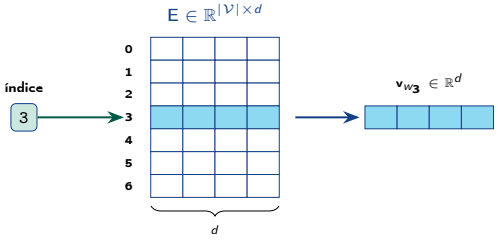
1. **Embeddings vectoriales** como capa de una red: `nn.Embedding`, `EmbeddingBag`, vectores preentrenados.
2. **Modelos de secuencias**: autorregresivos, Markov, modelos de lenguaje, perplejidad.
3. **Redes recurrentes** (RNN): estado oculto, entrenamiento con BPTT, gradientes que se desvanecen o explotan.
4. **LSTM, GRU**, RNN bidireccionales y multicapa.
5. Clasificación de AG_NEWS con RNN y LSTM (Lab 3.2).

La Capa de Embedding

Multiplicar un vector one-hot $e_i \in \{0,1\}^{|\mathcal{V}|}$ por una matriz $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ selecciona la fila i:

$$e_i^\top E = E[i,:] \in \mathbb{R}^d$$

- Una capa `Linear` sin sesgo sobre one-hot $\equiv$ **tabla de búsqueda**.
- `nn.Embedding(vocab_size, d)` recibe **índices** enteros y devuelve filas: no materializa el one-hot.
- E se entrena por retropropagación como cualquier peso, o se inicializa con vectores preentrenados.
- $d \ll |\mathcal{V}|$: de 95 810 dimensiones (BoW de AG_NEWS) a 32–300.



Una oración de T tokens se convierte en una matriz $T \times d$: **una fila por posición**, el orden se conserva.

Clasificador con Embeddings (Lab 3.4)

```
class EmbedClassifier(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim,
                 num_class):
        super().__init__()
        self.embedding = torch.nn.Embedding(
                            vocab_size, embed_dim)
        self.fc = torch.nn.Linear(embed_dim,
                                  num_class)

    def forward(self, x):           # x: (B, T)
        x = self.embedding(x)       # (B, T, d)
        x = torch.mean(x, dim=1)    # (B, d)
        return self.fc(x)           # (B, 4)
```

```
def padify(b):   # rellena con 0 hasta el maximo
    v = [encode(x[1]) for x in b]
    l = max(map(len, v))
    return (torch.LongTensor([t[0]-1 for t in b]),
            torch.stack([F.pad(torch.tensor(t),
                (0, l-len(t)), value=0) for t in v]))
```

Arquitectura

$$\hat{y} = \mathrm{softmax}\left(W \frac{1}{T} \sum_{t=1}^{T} v_{w_t} + b\right)$$

El promedio de embeddings es una "BoW densa": sigue **ignorando el orden**.

Resultado (lab)

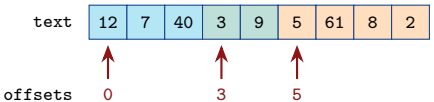
$d = 32$, 25 000 ejemplos: exactitud $\approx 76\%$ (frente a $\approx 86\%$ de BoW con 95 810 entradas). Con más épocas se acerca a $\sim$90%.

Relleno

Los minilotes necesitan secuencias del mismo largo: se rellenan con el índice 0. Los ceros *sí*

Secuencias de Largo Variable: `EmbeddingBag`

En lugar de rellenar, se concatenan todas las secuencias del lote en un **único vector** y se guardan los **desplazamientos** (*offsets*) donde empieza cada una.




```
self.embedding = torch.nn.EmbeddingBag(
        vocab_size, embed_dim)   # mode='mean'
def forward(self, text, off):
    return self.fc(self.embedding(text, off))
```

```
def offsetify(b):
    x = [torch.tensor(encode(t[1])) for t in b]
    o = [0] + [len(t) for t in x]
    o = torch.tensor(o[:-1]).cumsum(dim=0)
    return (torch.LongTensor(
                [t[0]-1 for t in b]),
            torch.cat(x),  # texto concatenado
            o)             # inicios
```

- `EmbeddingBag` = búsqueda + agregación (`mean`, `sum`, `max`) en una sola operación.
- Sin relleno: no se desperdicia memoria ni cómputo, y el relleno no contamina el promedio.
- Exactitud similar ($\approx 75\%$ en 25 000 ejemplos).

Embeddings Preentrenados

Los embeddings aprendidos sólo para clasificar no tienen por qué reflejar **significado**. Se pueden importar vectores entrenados con word2vec o GloVe sobre corpus enormes (clase 3.1):

```
import gensim.downloader as api
w2v = api.load('word2vec-google-news-300')
w2v.most_similar('neural')
# neuronal 0.78, neurons 0.73,
# neural_circuits 0.73, neuron 0.72, ...
```

```
E = torch.zeros(vocab_size, 300)
for i, w in enumerate(vocab.get_itos()):
    if w in w2v: E[i] = torch.tensor(w2v[w])
    else:        E[i] = torch.randn(300) * 0.1
net.embedding.weight.data = E
net.embedding.weight.requires_grad = False
```

Decisiones de diseño

- **Congelar** E: pocos datos etiquetados, se conserva la semántica general.
- **Ajustar** E: muchos datos, se adapta al dominio (riesgo de olvidar).
- Palabras fuera del vocabulario preentrenado: inicialización aleatoria o subpalabras (fastText).
- El tokenizador debe ser **compatible** con el del modelo preentrenado (minúsculas, frases unidas por _).

Límite

Con embeddings estáticos y un promedio, la red sabe *qué* palabras hay pero no *en qué orden*. Necesitamos un modelo de **secuencias**.

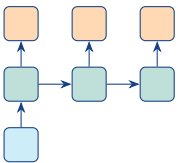
Datos Secuenciales

Hasta ahora las entradas tenían **largo fijo** (tablas, imágenes de 28×28). Texto, audio, video y series de tiempo son **secuencias** $x_1, \ldots, x_T$ de largo variable, con $x_t \in \mathbb{R}^d$.

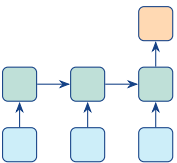
- Las secuencias completas se suponen i.i.d.; los elementos *dentro* de una secuencia son dependientes.
- La secuencia puede estar en la entrada, en la salida o en ambas.

Tipos de tareas

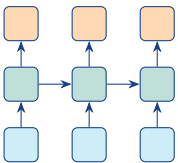
Forma	Ejemplo
fijo → secuencia	descripción de imágenes
secuencia → fijo	sentimiento, tópico (AG_NEWS)
secuencia → secuencia alineada	etiquetado POS, NER
secuencia → secuencia no alineada	traducción, diálogo



fijo → sec.



sec. → fijo



sec. alineada



Por la regla de la cadena, cualquier distribución conjunta se factoriza como

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1}, \ldots, x_1)$$

Problema: el número de entradas del condicional **crece con** t. Dos soluciones:

1. Ventana fija (Markov de orden τ)

Condicionar sólo en $x_{t-1}, \ldots, x_{t-\tau}$. Con $\tau = 1$:

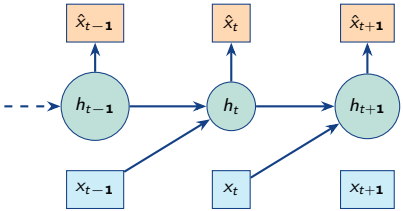
$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})$$

2. Modelo autorregresivo latente

Mantener un **resumen** h_t del pasado y actualizarlo:

$$\hat{x}_t = P(x_t \mid h_t), \qquad h_t = g(h_{t-1}, x_{t-1})$$

h_t nunca se observa: es **latente**. Esta es la idea de las RNN.



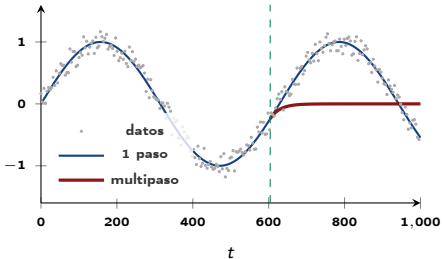
Predicción a k Pasos: Los Errores se Acumulan

Experimento d2l: $x_t = \sin(0.01\,t) + 0.2\,\varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0,1)$, $T = 1000$. Regresión lineal con $\tau = 4$ entradas, entrenada en los primeros 600 puntos.

- **Un paso**: usar siempre valores observados. Excelente en toda la serie.
- **Varios pasos**: desde $t = 604$ alimentar el modelo con sus propias predicciones:

$$\hat{x}_{605} = f(x_{601}, x_{602}, x_{603}, x_{604})$$
$$\hat{x}_{606} = f(x_{602}, x_{603}, x_{604}, \hat{x}_{605})$$
$$\hat{x}_{609} = f(\hat{x}_{605}, \hat{x}_{606}, \hat{x}_{607}, \hat{x}_{608})$$

- La predicción decae a una constante en pocos pasos.



Esquema cualitativo, adaptado de d2l §9.1.

Lecciones

Interpolar es más fácil que extrapolar. Nunca entrenar con datos del futuro. El error en k pasos se compone: $\epsilon_{\mathbf{2}} = \bar{\epsilon} + c\,\epsilon_{\mathbf{1}}$.

Texto como Secuencia: *The Time Machine*

Corpus de d2l: *The Time Machine* (H. G. Wells), poco más de 30 000 palabras.

1. Eliminar todo lo que no sea letra y pasar a minúsculas:
   ```
   re.sub('[^A-Za-z]+', ' ', text).lower()
   ```
2. Tokenizar a nivel de **carácter**.
3. Vocabulario: <unk>, espacio y 26 letras $\Rightarrow |\mathcal{V}| = 28$.
4. Corpus: **173 428** índices en una sola lista.

Palabras vs. caracteres

Palabras: vocabulario grande, secuencias cortas. Caracteres: vocabulario mínimo, secuencias largas. Subpalabras (BPE): punto medio.

Frecuencias (a nivel de palabra)

Unigramas		Trigramas	
the	2261	the time traveller	59
i	1267	the time machine	30
and	1245	the medical man	24
of	1155	it seemed to	16
a	816	it was a	15

Ley de Zipf

$$n_i \propto \frac{1}{i^\alpha} \iff \log n_i = -\alpha \log i + c$$

También vale para bigramas y trigramas. Consecuencia: la mayoría de los *n*-gramas son **raros**, y contar no basta.

Modelos de Lenguaje

Un **modelo de lenguaje** estima $P(x_1, \ldots, x_T)$ y genera texto muestreando $x_t \sim P(x_t \mid x_{t-1}, \ldots, x_1)$.

$P(\text{deep, learning, is, fun}) = P(\text{deep})\, P(\text{learning} \mid \text{deep})\, P(\text{is} \mid \text{deep, learning})\, P(\text{fun} \mid \ldots)$

Modelos de n-gramas

unigrama	$P(x_1)P(x_2)P(x_3)P(x_4)$
bigrama	$P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_2)P(x_4 \mid x_3)$
trigrama	$P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2)P(x_4 \mid x_2, x_3)$

Estimación por conteo:

$$\hat{P}(\text{learning} \mid \text{deep}) = \frac{n(\text{deep, learning})}{n(\text{deep})}$$

Suavizado de Laplace

n: total de palabras; m: palabras distintas.

$$\hat{P}(x) = \frac{n(x) + \epsilon_1/m}{n + \epsilon_1}, \quad \hat{P}(x' \mid x) = \frac{n(x, x') + \epsilon_2 \hat{P}(x')}{n(x) + \epsilon_2}$$

Con $\epsilon_1 = 0$ no hay suavizado; con $\epsilon_1 \to \infty$, $\hat{P}(x) \to 1/m$.

Limitaciones del conteo

- Ley de Zipf: casi todos los n-gramas largos tienen conteo 0.
- Para un contexto de $n-1$ tokens, el número de parámetros crece como $|\mathcal{V}|^n$.
- No hay noción de similitud: "gato" y "felino" son independientes.

Perplejidad

Un buen modelo asigna **alta probabilidad** a los tokens que realmente ocurren. Entropía cruzada promedio sobre n tokens:

$$\frac{1}{n}\sum_{t=1}^{n} -\log P(x_t \mid x_{t-1}, \ldots, x_1)$$

$$\mathrm{PPL} = \exp\Big(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t \mid x_{t-1}, \ldots, x_1)\Big)$$

Interpretación: el número efectivo de opciones entre las que el modelo "duda" en cada paso.

Valores de referencia

Modelo	PPL
Perfecto (prob. 1)	1
Uniforme sobre $\mathcal{V}$	$\lvert\mathcal{V}\rvert$
Prob. 0 a un token real	∞

Con caracteres, $|\mathcal{V}| = 28$: un modelo útil debe tener PPL muy por debajo de 28.

Ejemplo cualitativo

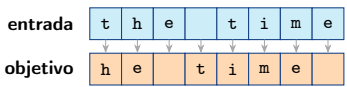
Continuar "It is raining. . .":
outside (buen modelo) · *banana tree* (algo de estructura) · *piouw;kcj pwepoiut* (modelo pobre)

Particionar una Secuencia en Minilotes

El corpus es una sola secuencia de T índices. Para entrenar un modelo de lenguaje:

- Cortar subsecuencias de largo n (`num_steps`).
- El **objetivo** es la misma ventana desplazada **un token**: $x_t \rightarrow x_{t+1}$.
- Un desplazamiento inicial aleatorio $d \in [0, n)$ en cada época varía los cortes.
- Un minilote reúne `batch_size` subsecuencias.

En d2l: `X = array[:, :-1]`, `Y = array[:, 1:]` sobre ventanas de largo `num_steps+1`.



Ejemplo d2l, ejercicio de juguete

Fragmento "machine", lote 1: entrada `machin` → objetivo `achine`. En cada paso se predice el siguiente carácter con softmax + entropía cruzada.

De MLP a RNN: El Estado Oculto

Aproximar todo el historial con un **estado oculto**:

$$P(x_t \mid x_{t-1}, \ldots, x_1) \approx P(x_t \mid h_{t-1}),$$
$$h_t = f(x_t, h_{t-1})$$

MLP (sin estado)

$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{\mathrm{xh}} + \mathbf{b}_{\mathrm{h}}), \quad \mathbf{O} = \mathbf{H}\mathbf{W}_{\mathrm{hq}} + \mathbf{b}_{\mathrm{q}}$

Cada entrada se procesa de forma independiente: sin memoria.

RNN

$$\mathbf{H}_t = \phi(\mathbf{X}_t\mathbf{W}_{\mathrm{xh}} + \mathbf{H}_{t-1}\mathbf{W}_{\mathrm{hh}} + \mathbf{b}_{\mathrm{h}})$$
$$\mathbf{O}_t = \mathbf{H}_t\mathbf{W}_{\mathrm{hq}} + \mathbf{b}_{\mathrm{q}}$$

Dimensiones (lote de n)

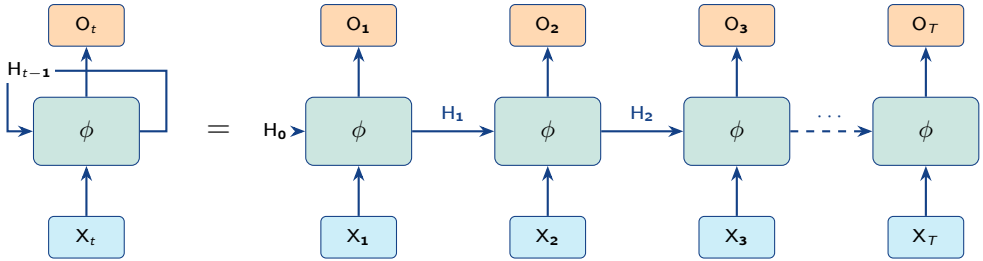
$\mathbf{X}_t \in \mathbb{R}^{n\times d}$	$\mathbf{W}_{\mathrm{xh}} \in \mathbb{R}^{d\times h}$	$\mathbf{b}_{\mathrm{h}} \in \mathbb{R}^{\mathbf{1}\times h}$
$\mathbf{H}_t \in \mathbb{R}^{n\times h}$	$\mathbf{W}_{\mathrm{hh}} \in \mathbb{R}^{h\times h}$	
$\mathbf{O}_t \in \mathbb{R}^{n\times q}$	$\mathbf{W}_{\mathrm{hq}} \in \mathbb{R}^{h\times q}$	$\mathbf{b}_{\mathrm{q}} \in \mathbb{R}^{\mathbf{1}\times q}$

- Los **mismos parámetros** en todos los pasos: su número no crece con el largo de la secuencia.
- Equivale a concatenar entrada y estado:

$$\mathbf{X}_t\mathbf{W}_{\mathrm{xh}} + \mathbf{H}_{t-\mathbf{1}}\mathbf{W}_{\mathrm{hh}} = [\mathbf{X}_t, \mathbf{H}_{t-\mathbf{1}}] \begin{bmatrix} \mathbf{W}_{\mathrm{xh}} \\ \mathbf{W}_{\mathrm{hh}} \end{bmatrix}$$

- ϕ suele ser tanh.

RNN Desenrollada en el Tiempo



mismos W_{xh}, W_{hh}, W_{hq} en cada paso

Modelo de lenguaje de caracteres

X_t: one-hot del carácter t ($d = |\mathcal{V}| = 28$); O_t: logits sobre el siguiente carácter ($q = |\mathcal{V}|$). Pérdida: entropía cruzada en cada paso.

Clasificación de una secuencia

Se ignoran las salidas intermedias y se clasifica con H_T (o el promedio de los H_t). Es el caso de AG_NEWS.

RNN desde Cero

```
class RNNScratch(nn.Module):
    def __init__(self, num_inputs, num_hiddens,
                 sigma=0.01):
        super().__init__()
        self.W_xh = nn.Parameter(
            torch.randn(num_inputs, num_hiddens)*sigma)
        self.W_hh = nn.Parameter(
            torch.randn(num_hiddens, num_hiddens)*sigma)
        self.b_h = nn.Parameter(
            torch.zeros(num_hiddens))

    def forward(self, inputs, state=None):
        # inputs: (num_steps, batch, num_inputs)
        if state is None:
            state = torch.zeros((inputs.shape[1],
                                 self.W_hh.shape[0]))
        outputs = []
        for X in inputs:           # bucle en el tiempo
            state = torch.tanh(X @ self.W_xh +
                               state @ self.W_hh + self.b_h)
            outputs.append(state)
        return outputs, state
```

Configuración d2l (Time Machine)

`batch_size`	1024
`num_steps`	32
`num_hiddens`	32
tasa de aprendizaje	1
épocas	100
recorte de gradiente θ	1

Generar texto

Calentar el estado con un prefijo y luego tomar el carácter más probable, realimentarlo y repetir:

Antes de entrenar: `it hasoadd dd dd dd dd dd`
Después: `it has and the trave the t`

Retropropagación a Través del Tiempo (BPTT)

Modelo simplificado: $h_t = f(x_t, h_{t-1}, w_h)$, $o_t = g(h_t, w_o)$, $L = \frac{1}{T}\sum_t l(y_t, o_t)$.

$$\frac{\partial L}{\partial w_h} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$$

El término difícil es recursivo, porque h_{t-1} también depende de w_h:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}$$

Desenrollando:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f_t}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^{t} \frac{\partial f_j}{\partial h_{j-1}} \right) \frac{\partial f_i}{\partial w_h}$$

El producto largo

El aporte del paso i al gradiente en t pasa por un **producto de $t-i$ jacobianos**. Con secuencias de cientos o miles de pasos:

- factores $< 1 \Rightarrow$ **se desvanece**;
- factores $> 1 \Rightarrow$ **explota**.

Caso lineal

Con $\mathrm{h}_t = \mathrm{W}_{\mathrm{hx}}\mathrm{x}_t + \mathrm{W}_{\mathrm{hh}}\mathrm{h}_{t-1}$:

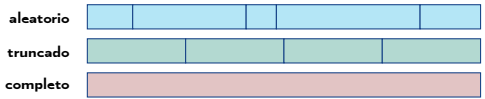
$$\frac{\partial L}{\partial \mathrm{h}_t} = \sum_{i=t}^{T} \left(\mathrm{W}_{\mathrm{hh}}^{\top}\right)^{T-i} \mathrm{W}_{\mathrm{qh}}^{\top} \frac{\partial L}{\partial \mathrm{o}_{T+t-i}}$$

Potencias de $\mathrm{W}_{\mathrm{hh}}^{\top}$: valores propios $|\lambda| < 1$ se anulan, $|\lambda| > 1$ divergen.

Truncamiento y Recorte de Gradientes

Estrategias de BPTT

1. **Completo**: sumar todos los términos. Lento e inestable; casi nunca se usa.
2. **Truncado**: cortar la suma tras τ pasos (en la práctica, `num_steps` y `detach()` del estado). Aproximado pero estable; actúa como regularizador hacia dependencias cortas.
3. **Aleatorio**: reemplazar el término recursivo por ξ_t con $P(\xi_t = 0) = 1 - \pi_t$, $P(\xi_t = \pi_t^{-1}) = \pi_t$, así $\mathbb{E}[\xi_t] = 1$. En la práctica no supera al truncado.



Recorte de gradientes

Contra la **explosión**: proyectar el gradiente (de todos los parámetros concatenados) a una bola de radio θ:

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right) \mathbf{g}$$

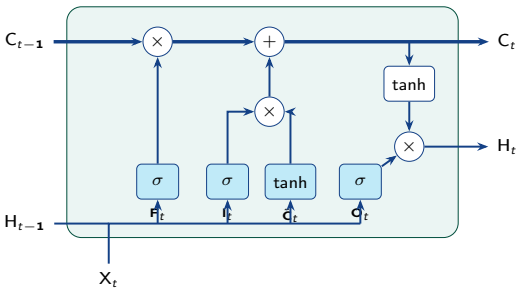
Conserva la dirección, acota la magnitud. En PyTorch:

```
torch.nn.utils.clip_grad_norm_(net.parameters(),
1)
```

Pero. . .

El recorte **no** resuelve el **desvanecimiento**: la RNN simple olvida lo que vio hace muchos pasos. Solución: **compuertas** (LSTM, GRU).

Long Short-Term Memory (LSTM)



Dos estados: la **celda de memoria** C_t y el estado oculto H_t. Tres compuertas con sigmoide (valores en $[0, 1]$, actúan como máscaras):

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \quad \text{olvido}$$
$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \quad \text{entrada}$$
$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \quad \text{salida}$$
$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$
$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$
$$H_t = O_t \odot \tanh(C_t)$$

La actualización de C_t es **aditiva**: si $F_t \approx 1$, el gradiente fluye sin atenuarse.

Intuición de las Compuertas y la GRU

La memoria como banderas

Las componentes de C_t pueden verse como *banderas* que se encienden y apagan:

- Al leer "*Alice*": encender "sujeto femenino".
- Al leer "*and Tom*": encender "sujeto plural".
- Al leer "*not*": invertir la polaridad de lo que sigue.

La compuerta de **olvido** decide qué banderas mantener; la de **entrada**, qué escribir; la de **salida**, qué mostrar en H_t.

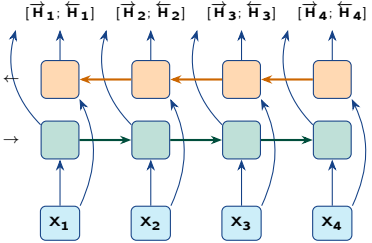
Gated Recurrent Unit (GRU)

Versión simplificada: un solo estado y dos compuertas.

$$
\begin{aligned}
R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\
Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \\
\tilde{H}_t &= \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\
H_t &= Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t
\end{aligned}
$$

R_t (reinicio) decide cuánto pasado usar; Z_t (actualización), cuánto copiar. Menos parámetros que LSTM, desempeño similar.

RNN Bidireccionales y Multicapa



- Dos RNN: una lee de izquierda a derecha, otra al revés. Se concatenan sus estados.
- Útil cuando toda la secuencia está disponible (clasificación, etiquetado). **No** sirve para generar texto.
- `bidirectional=True`: la salida tiene tamaño $2h$.

RNN multicapa (apiladas)

La secuencia de salidas de una capa es la entrada de la siguiente:

$$\mathbf{H}_t^{(\ell)} = \phi\left(\mathbf{H}_t^{(\ell-1)}\mathbf{W}_{\mathrm{xh}}^{(\ell)} + \mathbf{H}_{t-1}^{(\ell)}\mathbf{W}_{\mathrm{hh}}^{(\ell)} + \mathbf{b}_{\mathrm{h}}^{(\ell)}\right)$$

con $\mathbf{H}_t^{(0)} = \mathbf{X}_t$. Como en las CNN, las capas altas capturan patrones de mayor nivel.

`num_layers=L`: el estado final `h` tiene forma $(L \cdot \mathrm{dirs}, B, h)$.

En PyTorch

```
nn.LSTM(embed_dim, hidden_dim, num_layers=2,
        bidirectional=True, batch_first=True)
```

Clasificador RNN y LSTM (Lab 3.2)

```
class RNNClassifier(nn.Module):
  def __init__(self, vocab_size, embed_dim,
               hidden_dim, num_class):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size,
                                  embed_dim)
    self.rnn = nn.RNN(embed_dim, hidden_dim,
                      batch_first=True)
    self.fc = nn.Linear(hidden_dim, num_class)

  def forward(self, x):          # (B, T)
    x = self.embedding(x)        # (B, T, d)
    x, h = self.rnn(x)           # x: (B, T, h)
    return self.fc(x.mean(dim=1))
```

```
class LSTMClassifier(nn.Module):
  # ... igual, pero:
  self.rnn = nn.LSTM(embed_dim, hidden_dim,
                     batch_first=True)
  def forward(self, x):
    x = self.embedding(x)
    x, (h, c) = self.rnn(x)
    return self.fc(h[-1])   # ultimo estado
```

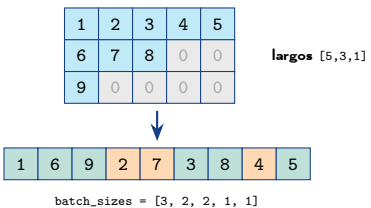
Salidas de `nn.RNN` / `nn.LSTM`

- `x`: estados en **todos** los pasos, (B, T, h).
- `h`: estado **final** por capa, (L, B, h).
- LSTM devuelve además la celda `c`.

Configuración: $d = 64$, $h = 32$, lote 16, Adam $\eta = 10^{-3}$, una época.

Secuencias Empaquetadas

Con relleno, la RNN ejecuta pasos sobre ceros que no aportan información, y en `h[-1]` el último estado corresponde al relleno, no al último token real.

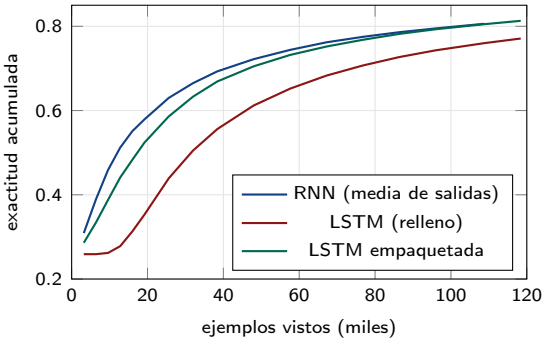


El paso 1 procesa un minilote de 3, los pasos 2–3 de 2, y los pasos 4–5 de 1.

```
def forward(self, x, lengths):
    x = self.embedding(x)
    px = nn.utils.rnn.pack_padded_sequence(
             x, lengths, batch_first=True,
             enforce_sorted=False)
    px, (h, c) = self.rnn(px)
    # opcional, si se usan las salidas:
    x, _ = nn.utils.rnn.pad_packed_sequence(
                px, batch_first=True)
    return self.fc(h[-1])
```

- El *collate* (`pad_length`) devuelve también los largos.
- `lengths` debe estar en **CPU**.
- `h[-1]` es ahora el estado tras el último token *real* de cada secuencia.

Resultados en AG_NEWS (una época)



- La LSTM con relleno tarda en arrancar ($\approx$ 26%, azar con 4 clases, durante los primeros 10 000 ejemplos): `h[-1]` ve sobre todo ceros de relleno.
- Empaquetar corrige eso: misma red, **81.4%** frente a 77.3%.
- La RNN simple con promedio de salidas llega a $\approx$ 80.6%.
- Todas quedan por debajo del BoW lineal (86%) tras una época: las RNN son **lentas de entrenar**; necesitan más épocas, GPU y tasa de aprendizaje pequeña.

Exactitud de entrenamiento acumulada, tomada de las salidas del notebook.

Conclusiones

- Una capa de **embedding** es una tabla de búsqueda entrenable ($\equiv$ one-hot $\times$ matriz). Puede inicializarse con word2vec/GloVe y congelarse o ajustarse.
- Promediar embeddings (`EmbeddingBag`) es rápido pero **ignora el orden**.
- Un **modelo de lenguaje** factoriza $P(x_1, \ldots, x_T)$ autorregresivamente; los n-gramas sufren de dispersión (Zipf). Se evalúa con **perplejidad**.
- Una **RNN** resume el pasado en un estado oculto H_t con parámetros compartidos en el tiempo, y se entrena con **BPTT**.
- Los productos de jacobianos causan gradientes que **explotan** (recorte) o **se desvanecen** (compuertas: **LSTM**, **GRU**).
- En la práctica: empaquetar secuencias, usar variantes bidireccionales y multicapa.

Para el laboratorio 3.2

Entrenar RNN, LSTM y LSTM empaquetada en AG_NEWS. Probar `bidirectional=True`, `num_layers=2` y una GRU; inicializar el embedding con vectores preentrenados y comparar contra el BoW de la clase 3.1.

Referencias

- A. Zhang, Z. C. Lipton, M. Li & A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023. Cap. 9: *Recurrent Neural Networks*; Cap. 10: *Modern Recurrent Neural Networks*. `d2l.ai`
- S. Hochreiter & J. Schmidhuber, "Long Short-Term Memory", *Neural Computation* 9(8), 1997.
- K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation" (GRU), EMNLP 2014.
- Y. Bengio, P. Simard & P. Frasconi, "Learning Long-Term Dependencies with Gradient Descent is Difficult", *IEEE Trans. Neural Networks*, 1994.
- R. Pascanu, T. Mikolov & Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks", ICML 2013.
- M. Schuster & K. K. Paliwal, "Bidirectional Recurrent Neural Networks", *IEEE Trans. Signal Processing*, 1997.
- C. Olah, "Understanding LSTM Networks", 2015. `colah.github.io/posts/2015-08-Understanding-LSTMs`
- T. Mikolov et al., "Distributed Representations of Words and Phrases and their Compositionality", NeurIPS 2013.